

CS422 Assignment - 3 Group - 6

Aman Dixit
230112

Aditya Gautam
220064

Abhijeet Agarwal
210025

Saaumitra Raaj
220928

April 21, 2026

Constant-Time AES-128 Implementation and Performance Analysis

Contents

1	Introduction and Objective	2
2	Implementation Details & Optimisations	2
2.1	Optimisation 1: 32-bit SWAR for MixColumns	2
2.2	Optimisation 2: 64-bit SWAR for SubBytes GF Arithmetic	3
2.3	Optimisation 3: GF(2^8) Squaring via Linear Bit-Map	3
2.4	Optimisation 4: Optimal Addition Chain for x^{254}	4
2.5	Optimisation 5: Interleaved Dual-Chain for ILP in SubBytes	5
2.6	Optimisation 6: SWAR Affine Transform in SubBytes	5
2.7	Optimisation 7: Zero-Overhead Data Marshalling via Macros	6
2.8	Summary of Optimisations	7
3	Performance Evaluation	8
3.1	Benchmark Results (Default / No Compiler Optimization)	8
3.2	Benchmark Results (-O4 Optimization)	8
3.3	Conclusion	8

1 Introduction and Objective

AES-128 is a widely used symmetric cryptographic primitive that processes a 128-bit plaintext and a 128-bit master key to produce a 128-bit ciphertext. Standard, high-performance software implementations of AES-128 rely heavily on pre-computed lookup tables (such as S-boxes and T-tables). While these tables provide significant speed advantages, they result in memory accesses that are intrinsically tied to the secret key. This architectural flaw makes the implementation vulnerable to cache-timing side-channel attacks, allowing an attacker to recover the master key by observing memory access patterns.

The objective of this assignment is to design and implement a secure AES-128 encryption engine that strictly avoids secret-dependent memory accesses. Our implementation entirely avoids lookup-table S-boxes and hardware-specific AES-NI instructions, achieving strict constant-time execution through dynamic mathematical calculations and architectural optimizations.

2 Implementation Details & Optimisations

This section details every performance optimisation applied to the custom AES-128 encryption implementation. The goal is a *table-lookup-free*, bitsliced/SWAR design that exploits modern 64-bit registers and out-of-order execution without requiring architecture-specific intrinsics (SSE, AES-NI, NEON, etc.).

The key ideas are:

- Pack multiple $\text{GF}(2^8)$ elements into one machine word and operate on all of them simultaneously (**SWAR** — *SIMD Within A Register*).
- Replace expensive general multiplications with cheaper squarings wherever the algebraic structure of the problem allows it.
- Choose an **optimal addition chain** for the inversion exponent x^{254} so that the total number of field operations is minimised.
- Write independent computation chains **interleaved** so that a superscalar, out-of-order CPU can execute them in parallel (**ILP** — *Instruction-Level Parallelism*).

2.1 Optimisation 1: 32-bit SWAR for MixColumns

Motivation: The naive MixColumns implementation processes one byte at a time, requiring separate $\text{GF}(2^8)$ `xtime` calls and XOR operations for every element of each column — approximately 20 scalar byte operations per column.

Technique: Four bytes of a column are packed into a single `uint32_t`. The $\text{GF}(2^8)$ doubling operation (`xtime`) is then applied to all four bytes in parallel using bitmask arithmetic:

```
1 static inline uint32_t xtime32(uint32_t x) {  
2     uint32_t msb = x & 0x80808080u;  
3     return ((x & 0x7F7F7F7Fu) << 1) ^ ((msb >> 7) * 0x1Bu);  
4 }
```

Listing 1: 32-bit SWAR `xtime`

- The mask `0x80808080` isolates the most-significant bit of each byte, detecting which lanes need reduction modulo $x^8 + x^4 + x^3 + x + 1$ (i.e. XOR with `0x1B`).
- The mask `0x7F7F7F7F` prevents the left-shift from bleeding the MSB of one lane into the LSB of the next.

The full MixColumns transform for one column is then reduced to five word-level operations:

```

1 uint32_t c      = (a0<<24)|(a1<<16)|(a2<<8)|a3;
2 uint32_t crot   = (a1<<24)|(a2<<16)|(a3<<8)|a0;
3 uint32_t res    = c
4                 ^ ((a0^a1^a2^a3) * 0x01010101u)
5                 ^ xtime32(c ^ crot);

```

Listing 2: SWAR MixColumns for one column

Benefit: Four independent $\text{GF}(2^8)$ doublings are collapsed into one 32-bit operation, reducing the per-column work from ~ 20 byte operations to ~ 5 word operations.

2.2 Optimisation 2: 64-bit SWAR for SubBytes GF Arithmetic

Motivation: SubBytes is the most expensive AES step because it requires computing the $\text{GF}(2^8)$ inverse for every one of the 16 state bytes. Operating one byte at a time wastes $7/8$ of each 64-bit register.

Technique: All $\text{GF}(2^8)$ primitives are lifted to 64-bit, processing **8 bytes simultaneously** inside a single `uint64_t`. Each byte occupies its own 8-bit lane; the lane-isolation masks prevent carry between lanes.

```

1 static inline uint64_t xtime64(uint64_t x) {
2     uint64_t msb = x & 0x8080808080808080ULL;
3     return ((x & 0x7F7F7F7F7F7F7F7FULL) << 1)
4           ^ ((msb >> 7) * 0x1BULL);
5 }

```

Listing 3: 64-bit SWAR xtime

General GF multiplication is implemented via the standard double-and-add algorithm, fully unrolled over all 8 bits so there is no scalar loop overhead:

```

1 static inline uint64_t gf_mul64(uint64_t a, uint64_t b) {
2     const uint64_t M = 0x0101010101010101ULL;
3     const uint64_t N = 0x7F7F7F7F7F7F7F7FULL;
4     uint64_t p = 0, lsb;
5
6     lsb = b & M; p ^= a & (lsb * 255ULL); a = xtime64(a); b = (b>>1)&N;
7     lsb = b & M; p ^= a & (lsb * 255ULL); a = xtime64(a); b = (b>>1)&N;
8     /* ... repeated for all 8 bits ... */
9     return p;
10 }

```

Listing 4: GF multiplication on 8 byte-lanes simultaneously

Benefit: One call to `gf_mul64` computes 8 independent $\text{GF}(2^8)$ multiplications. Compared to a scalar byte loop, this is an $8\times$ throughput improvement for the multiplication primitive.

2.3 Optimisation 3: $\text{GF}(2^8)$ Squaring via Linear Bit-Map

Motivation: The inversion addition chain requires far more squarings than general multiplications. Squaring should therefore be as fast as possible — ideally faster than `gf_mul64`.

Technique: Squaring in $\text{GF}(2^8)$ is a **$\text{GF}(2)$ -linear map**: because the field characteristic is 2,

$$\left(\sum_{i=0}^7 a_i x^i \right)^2 = \sum_{i=0}^7 a_i x^{2i}.$$

The high-degree terms $x^8, x^{10}, x^{12}, x^{14}$ are reduced modulo the AES irreducible polynomial $p(x) = x^8 + x^4 + x^3 + x + 1$:

$$\begin{aligned} x^8 &\equiv x^4 + x^3 + x + 1 \\ x^{10} &\equiv x^6 + x^5 + x^3 + x^2 \\ x^{12} &\equiv x^7 + x^5 + x^3 + x + 1 \\ x^{14} &\equiv x^7 + x^4 + x^3 + x \end{aligned}$$

This yields a fixed, precomputed Boolean circuit — each output bit is a small XOR of input bits:

$$\begin{aligned} \text{out}[0] &= a_0 \oplus a_4 \oplus a_6 \\ \text{out}[1] &= a_4 \oplus a_6 \oplus a_7 \\ \text{out}[2] &= a_1 \oplus a_5 \\ \text{out}[3] &= a_4 \oplus a_5 \oplus a_6 \oplus a_7 \\ \text{out}[4] &= a_2 \oplus a_4 \oplus a_7 \\ \text{out}[5] &= a_5 \oplus a_6 \\ \text{out}[6] &= a_3 \oplus a_5 \\ \text{out}[7] &= a_6 \oplus a_7 \end{aligned}$$

Applied in SWAR fashion across all 8 byte-lanes simultaneously:

```

1 static inline uint64_t gf_sq64(uint64_t x) {
2     const uint64_t M = 0x0101010101010101ULL;
3     uint64_t a0=(x >> 0)&M, a1=(x >> 1)&M, a2=(x >> 2)&M, a3=(x >> 3)&M;
4     uint64_t a4=(x >> 4)&M, a5=(x >> 5)&M, a6=(x >> 6)&M, a7=(x >> 7)&M;
5
6     uint64_t t46=a4^a6, t47=a4^a7, t56=a5^a6, t67=a6^a7;
7
8     uint64_t o0=a0^t46, o1=t46^a7, o2=a1^a5, o3=t47^t56;
9     uint64_t o4=a2^t47, o6=a3^a5;
10
11     return o0|(o1<<1)|(o2<<2)|(o3<<3)|(o4<<4)|(t56<<5)|(o6<<6)|(t67<<7);
12 }

```

Listing 5: GF squaring as a linear bit-map (8 lanes)

Benefit: Squaring requires **zero GF multiplications** — only bitwise shifts and XORs. It is significantly cheaper than `gf_mul64`, which is exploited heavily in the next optimisation.

2.4 Optimisation 4: Optimal Addition Chain for x^{254}

Motivation: The AES S-box requires $x^{-1} = x^{254}$ in $\text{GF}(2^8)$ (by Fermat’s little theorem: $x^{255} = 1$ for $x \neq 0$). A naive square-and-multiply implementation uses **11 general multiplications**. Multiplications are the most expensive operation, so minimising their count directly reduces SubBytes cost.

Technique: An **optimal addition chain** is chosen that exploits cheap squarings to reach exponent 254 with only **7 squarings + 4 multiplications**:

$$x \xrightarrow{\text{sq}} x^2 \xrightarrow{\times x} x^3 \xrightarrow{\text{sq}} x^6 \xrightarrow{\text{sq}} x^{12} \xrightarrow{\times x^2} x^{14} \xrightarrow{\times x} x^{15} \xrightarrow{\text{sq}} x^{30} \xrightarrow{\text{sq}} x^{60} \xrightarrow{\text{sq}} x^{120} \xrightarrow{\text{sq}} x^{240} \xrightarrow{\times x^{14}} x^{254}$$

```

1 static inline uint64_t gf_inv64(uint64_t x) {
2     uint64_t x2 = gf_sq64(x);
3     uint64_t x3 = gf_mul64(x2, x);
4     uint64_t x6 = gf_sq64(x3);

```

```

5  uint64_t x12 = gf_sq64(x6);
6  uint64_t x14 = gf_mul64(x12, x2);
7  uint64_t x15 = gf_mul64(x14, x);
8  uint64_t x30 = gf_sq64(x15);
9  uint64_t x60 = gf_sq64(x30);
10 uint64_t x120 = gf_sq64(x60);
11 uint64_t x240 = gf_sq64(x120);
12 return gf_mul64(x240, x14);
13 }

```

Listing 6: GF inversion via optimal addition chain

Benefit: Compared to 11 multiplications, the chain uses only 4 multiplications and 7 squarings. Since squarings are pure XOR/shift operations (much cheaper than `gf_mul64`), this yields approximately a **35% reduction** in SubBytes computation cost versus the prior implementation.

2.5 Optimisation 5: Interleaved Dual-Chain for ILP in SubBytes

Motivation: Even with the fast inversion chain, SubBytes must be applied to all 16 state bytes. Two independent 64-bit words (`h1` for columns 0–1, `h2` for columns 2–3) must each go through the same inversion sequence. Executing them sequentially leaves the CPU’s execution units idle.

Technique: The two inversion chains are written **interleaved** — alternating an `a-chain` instruction with a `b-chain` instruction. Since `a` and `b` are *entirely data-independent*, an out-of-order CPU can **execute both chains in parallel**:

```

1  uint64_t a2 = gf_sq64(a);      uint64_t b2 = gf_sq64(b);
2  uint64_t a3 = gf_mul64(a2, a); uint64_t b3 = gf_mul64(b2, b);
3  uint64_t a6 = gf_sq64(a3);      uint64_t b6 = gf_sq64(b3);
4  uint64_t a12 = gf_sq64(a6);      uint64_t b12 = gf_sq64(b6);
5  uint64_t a14 = gf_mul64(a12, a2); uint64_t b14 = gf_mul64(b12, b2);
6  uint64_t a15 = gf_mul64(a14, a); uint64_t b15 = gf_mul64(b14, b);
7  uint64_t a30 = gf_sq64(a15);      uint64_t b30 = gf_sq64(b15);
8  /* ... and so on ... */

```

Listing 7: Interleaved dual inversion chains in `subbytes_128()`

Benefit: On a superscalar out-of-order processor, the two chains can be overlapped, effectively **halving the latency** of the full 16-byte SubBytes operation compared to running them sequentially.

2.6 Optimisation 6: SWAR Affine Transform in SubBytes

Motivation: After GF inversion, each byte must undergo the AES affine transform, applying 4 rotations and 5 XORs *per byte*. Applied naively, this is heavily resource-intensive.

Technique: A per-byte left circular rotation is implemented in SWAR style using bitmask arithmetic so that all 8 byte-lanes are rotated simultaneously:

```

1  static inline uint64_t rot18_64(uint64_t x, int n) {
2      const uint64_t top =
3          0x0101010101010101ULL * ((0xFFULL << (8-n)) & 0xFFULL);
4      return ((x & ~top) << n) | ((x & top) >> (8-n));
5  }

```

Listing 8: SWAR per-byte circular rotation (8 lanes)

The constant `0x63` is broadcast across all 8 lanes as `C = 0x6363636363636363ULL` and XOR’d in one operation. The complete affine transform for all 8 bytes becomes five 64-bit instructions:

```

1  const uint64_t C = 0x6363636363636363ULL;
2  *h1 = ai ^ rotl8_64(ai,1) ^ rotl8_64(ai,2)
3         ^ rotl8_64(ai,3) ^ rotl8_64(ai,4) ^ C;

```

Listing 9: SWAR affine transform for 8 bytes at once

Benefit: The affine transform is applied to all 8 bytes in 5 word-level operations instead of $8 \times 9 = 72$ byte-level operations — an $\sim 14\times$ reduction in instruction count for this step.

2.7 Optimisation 7: Zero-Overhead Data Marshalling via Macros

Motivation: Feeding bytes into the 64-bit SWAR pipeline requires packing two columns (8 bytes) into a `uint64_t` and unpacking the result. Doing this with loops or intermediate arrays adds overhead.

Technique: Two macros encode the pack/unpack as **single compile-time expressions** with no loops, no temporary arrays, and no memory allocation:

```

1  #define PACK_H(s,c0,c1) ( \
2      ((uint64_t)(s)[0][c1]<<56)| \
3      ((uint64_t)(s)[1][c1]<<48)| \
4      ((uint64_t)(s)[2][c1]<<40)| \
5      ((uint64_t)(s)[3][c1]<<32)| \
6      ((uint64_t)(s)[0][c0]<<24)| \
7      ((uint64_t)(s)[1][c0]<<16)| \
8      ((uint64_t)(s)[2][c0]<< 8)| \
9      (uint64_t)(s)[3][c0]) \
10
11 #define UNPACK_H(h,s,c0,c1) do { \
12     (s)[0][c0]=(h>>24)&0xFF; (s)[1][c0]=(h>>16)&0xFF; \
13     (s)[2][c0]=(h>> 8)&0xFF; (s)[3][c0]= h      &0xFF; \
14     (s)[0][c1]=(h>>56)&0xFF; (s)[1][c1]=(h>>48)&0xFF; \
15     (s)[2][c1]=(h>>40)&0xFF; (s)[3][c1]=(h>>32)&0xFF; \
16 } while(0)

```

Listing 10: PACK and UNPACK macros for zero-overhead marshalling

Benefit: Data layout conversion between the column-major state array and the SWAR 64-bit representation incurs **zero runtime overhead** beyond the unavoidable byte reads and writes.

2.8 Summary of Optimisations

Optimisation		Technique	Benefit
MixColumns		32-bit SWAR	4 bytes processed per word-level operation; $\sim 4\times$ fewer instructions per column
SubBytes arithmetic	GF	64-bit SWAR	8 bytes processed per operation; $8\times$ throughput on GF primitives
GF squaring		Linear bit-map (pre-computed Boolean circuit)	No multiplications; pure XOR/shift — much cheaper than <code>gf_mul64</code>
GF inversion		Optimal addition chain (7 sq + 4 mul)	$\sim 35\%$ faster SubBytes vs. 11-multiplication baseline
SubBytes dual-chain		Interleaved a/b chains	Exploits out-of-order / superscalar execution; effectively halves SubBytes latency
Affine transform		SWAR rotations + broadcast constant	8 bytes per op; $\sim 14\times$ fewer instructions vs. byte loop
Data marshalling		PACK/UNPACK macros	Zero-overhead layout conversion; compiler-optimised into minimal shift/OR sequences

Table 1: Summary of all optimisations applied to the custom AES-128 implementation.

3 Performance Evaluation

We analyzed the performance of our custom constant-time implementation against the highly optimized OpenSSL AES baseline. The performance was profiled using a dynamic binary instrumentation tool (Intel Pin) to count instructions by category. As specified by the assignment guidelines, memory accesses were weighted at 5 clock cycles, while all other ALU/logical instructions were weighted at 1 clock cycle (yielding the formula: $\text{Total Cycles} = (4 * \text{Memory Accesses}) + \text{Total Instructions}$).

Running the leakage testing script on the OpenSSL implementation predictably fails because its lookup tables introduce secret-dependent memory accesses. However, it serves as our performance reference. We benchmarked the code both with default compiler settings and with aggressive optimizations (-O4) via GCC to maximize instruction-level parallelism.

3.1 Benchmark Results (Default / No Compiler Optimization)

Metric	OpenSSL	Custom	Ratio (Custom/OpenSSL)
Total Instructions	502,362	548,335	
Memory Accesses	168,363	195,896	
ALU / Logic Instructions	333,999	352,439	
Total Run Time (Cycles)	1,175,814	1,331,919	1.13x
Average CPI	2.34	2.43	

Table 2: Instruction Mix and Cycle Analysis (Without -O4)

3.2 Benchmark Results (-O4 Optimization)

Metric	OpenSSL	Custom	Ratio (Custom/OpenSSL)
Total Instructions	502,170	521,382	
Memory Accesses	168,222	168,589	
ALU / Logic Instructions	333,948	352,793	
Total Run Time (Cycles)	1,175,058	1,195,738	1.02x
Average CPI	2.34	2.29	

Table 3: Instruction Mix and Cycle Analysis (With -O4)

3.3 Conclusion

Our software-only, constant-time AES-128 implementation achieved remarkable performance parity with the OpenSSL table-based baseline. Without relying on compiler optimizations, our manual architectural and mathematical enhancements (SWAR, optimal addition chains, linear-map squaring) restricted the performance overhead to just **13.2%** (1,331,919 cycles vs 1,175,814 cycles).

When paired with aggressive -O4 compiler optimizations to fully exploit instruction-level parallelism and optimal register allocation, the execution time of our custom implementation dropped to **1,195,738 cycles**. This reduced the performance penalty to a mere **1.76%** overhead compared to OpenSSL. This proves that strict constant-time cryptographic security can be achieved dynamically without sacrificing software performance or relying on specialized hardware instructions.

References

- Assignment 3: Constant-Time AES Implementation
- Intel PIN Tool Documentation